

WordPress 研修

エラーハンドリング解説書

全10問 トラブルシューティング手順・解説

構成: Apache (Web) + PHP-FPM (AP) + MariaDB (DB)

環境: Docker Compose (Amazon Linux 2023)

ネットワーク: 10.10.10.0/24 固定IP

Web: 10.10.10.11 **AP:** 10.10.10.12 **DB:** 10.10.10.13

リポジトリ: github.com/kazuma-kawagoe/Wordpress-training

各問は Git ブランチ (lv1-q1 ~ lv1-q10) で管理されています

`git checkout lv1-qN` → `docker compose up -d --build` で環境起動

目次

問1：Apache（httpd）が起動していない	サービス管理
問2：php.conf のプロキシポートが間違っている	Web設定
問3：wp-config.php の DB名誤り+構文エラー	AP設定
問4：php.conf で PHP へのアクセスが全拒否	Web設定
問5：php.conf のタイムアウト値が極端に短い	Web設定
問6：MariaDB の bind-address が localhost のみ	DB設定
問7：wp-config.php のパーミッションが 000	パーミッション
問8：wp-config.php の DB_HOST が存在しない IP	AP設定
問9：PHP-FPM の allowed_clients が間違っている	AP設定
問10：DB ユーザーに GRANT 権限がない	DB設定

共通のエラーハンドリング手順（リクエストフロー順）

WordPress 3層構成のトラブルシューティングは、リクエストの流れに沿って上流から順に確認するのが鉄則です。

順序	対象	確認コマンド	確認ポイント
Step 0	ブラウザ	<code>http://localhost:8080</code>	症状の確認（503/500/403/接続拒否）
Step 1	Web (10.10.10.11)	<code>docker exec -it web bash</code>	httpd 起動確認、設定構文、エラーログ
Step 2	AP (10.10.10.12)	<code>docker exec -it ap bash</code>	php-fpm 起動確認、listen設定、wp-config.php
Step 3	DB (10.10.10.13)	<code>docker exec -it db bash</code>	mariadb 起動確認、bind-address、権限

よく使うコマンド一覧:

目的	コマンド
----	------

サービス状態確認	<code>systemctl status httpd / php-fpm / mariadb</code>
プロセス確認	<code>ps aux grep httpd</code>
ポート確認	<code>ss -tlnp grep ポート番号</code>
Apache エラーログ	<code>tail -20 /var/log/httpd/error_log</code>
設定構文チェック	<code>httpd -t</code>
DB 接続テスト	<code>mysql -u ユーザー -h ホスト -p DB名</code>
ファイル権限確認	<code>ls -la ファイルパス</code>

問1：Apache (httpd) が起動していない ★☆☆

カテゴリ: サービス管理 | 対象サーバ: Web (10.10.10.11) | ブランチ: lv1-q1

▶ ブラウザでの症状

http://localhost:8080 にアクセスすると「接続が拒否されました」または応答なし

▶ 原因

web コンテナの Dockerfile から `RUN systemctl enable httpd` が削除されている。コンテナ起動時に httpd が自動起動しないため、ポート 80 で何もリッスンしていない。

```
- RUN systemctl enable httpd ← この行が削除されている
```

エラーハンドリング手順

1 Web サーバに入る

```
$ docker exec -it web bash
```

2 httpd の状態を確認する

```
# systemctl status httpd  
# → Active: inactive (dead) と表示される
```

3 httpd を起動する

```
# systemctl start httpd
```

4 起動を確認する

```
# systemctl status httpd  
# → Active: active (running) になっていればOK  
  
# ss -tlnp | grep 80  
# → ポート80でリッスンしていることを確認
```

5 ブラウザで再確認 → WordPress のセットアップ画面が表示されれば正解

▶ 恒久的な修正方法

Dockerfile に `RUN systemctl enable httpd` を追記して再ビルドする。

▶ 学びポイント

「そもそもサービスが動いているか？」はトラブルシューティングの最初の一步。 `systemctl status` と `ss -tlnp` で「起動しているか」「ポートで待ち受けているか」を確認する癖をつけよう。

問2：php.conf のプロキシポートが間違っている ★☆☆

カテゴリ: Web設定 (php.conf) | 対象サーバ: Web (10.10.10.11) | ブランチ: lv1-q2

▶ ブラウザでの症状

503 Service Unavailable

▶ 原因

php.conf 内の `SetHandler` と `Proxy` のポート番号が `8000` (正) → `9000` (誤) に変更されている。Apache が PHP-FPM に接続しようとするが、PHP-FPM は `8000` で待ち受けているため接続拒否される。

```
- SetHandler "proxy:fcgi://10.10.10.12:9000"  
+ SetHandler "proxy:fcgi://10.10.10.12:8000"  
  
- <Proxy "fcgi://10.10.10.12:9000">  
+ <Proxy "fcgi://10.10.10.12:8000">
```

エラーハンドリング手順

1 Web サーバに入ってエラーログを確認

```
$ docker exec -it web bash  
# tail -20 /var/log/httpd/error_log  
# → "Connection refused" や "proxy:fcgi" に関するエラーが見える
```

2 php.conf の内容を確認

```
# cat /etc/httpd/conf.d/php.conf  
# → ポート番号が 9000 になっていることに気づく
```

3 AP サーバで PHP-FPM が実際にリッスンしているポートを確認

```
$ docker exec -it ap bash  
# ss -tlnp | grep php  
# → 0.0.0.0:8000 でリッスンしている → php.conf の 9000 が間違い
```

4 php.conf のポートを修正

```
# vi /etc/httpd/conf.d/php.conf  
# 9000 → 8000 に修正 (2箇所)
```

5 Apache を再起動して確認

```
# systemctl restart httpd
```

▶ 学びポイント

503 エラーは「バックエンドに接続できない」ことを意味する。Apache のエラーログには接続先の IP:ポート が出るので、実際にそのポートで何がリスンしているかを `ss -tlnp` で確認するのが定石。

問3：wp-config.php の DB名誤り+構文エラー ★★☆☆

カテゴリ: AP設定 (wp-config.php) | 対象サーバ: AP (10.10.10.12) | ブランチ: lv1-q3

▶ ブラウザでの症状

500 Internal Server Error または「Error establishing a database connection」

▶ 原因（2箇所）

- ① DB名が `wordpress-db`（正）→ `wordpress-database`（誤）に変更
- ② `define( 'DB_NAME', 'wordpress-database' )` の行末セミコロンが欠落（構文エラー）

```
- define( 'DB_NAME', 'wordpress-database' ) ← DB名が違う+セミコロンなし  
+ define( 'DB_NAME', 'wordpress-db' ); ← 正しいDB名+セミコロンあり
```

エラーハンドリング手順

1 Web のエラーログを確認

```
$ docker exec -it web bash  
# tail -20 /var/log/httpd/error_log  
# → 500 エラーまたは PHP のパースエラーが出ている可能性
```

2 AP サーバで wp-config.php を確認

```
$ docker exec -it ap bash  
# cat /var/www/html/wp-config.php  
# → DB_NAME の値と構文をチェック
```

3 DB 側で実際のデータベース名を確認

```
$ docker exec -it db bash  
# mysql -u root -e "SHOW DATABASES;"  
# → wordpress-db が正しい名前
```

4 wp-config.php を修正

```
# vi /var/www/html/wp-config.php  
# DB_NAME を 'wordpress-db' に修正  
# 行末にセミコロン ; を追加
```

▶ 学びポイント

PHP の構文エラー（セミコロン欠落）は画面に何も表示されない白い画面（WSOD: White Screen of Death）になることがある。 `php -l ファイル名` で構文チェックできる。また、DB名の不一致は `SHOW DATABASES;` で実際の名前を確認するのが確実。

問4：php.conf で PHP へのアクセスが全拒否 ★☆☆

カテゴリ: Web設定 (php.conf) | 対象サーバ: Web (10.10.10.11) | ブランチ: lv1-q4

▶ ブラウザでの症状

403 Forbidden

▶ 原因

php.conf の `<FilesMatch>` ブロック内に `Require all denied` が追加されている。全ての .php ファイルへのアクセスが拒否される。

```
<FilesMatch \.(php|phar)$>
    SetHandler "proxy:fcgi://10.10.10.12:8000"
    Require all denied ← この行が追加されている
</FilesMatch>
```

エラーハンドリング手順

1 Web サーバのエラーログを確認

```
# tail -20 /var/log/httpd/error_log
# → "client denied by server configuration" のようなメッセージ
```

2 php.conf を確認

```
# cat /etc/httpd/conf.d/php.conf
# → Require all denied を発見
```

3 該当行を削除またはコメントアウト

```
# vi /etc/httpd/conf.d/php.conf
# Require all denied の行を削除
```

4 Apache を再起動

```
# systemctl restart httpd
```

▶ 学びポイント

403 は「アクセス権がない」ことを意味する。Apache の `Require` ディレクティブによるアクセス制御を理解しよう。 `Require all granted`（全許可）と `Require all denied`（全拒否）の違いは基本中の基本。

問5：php.conf のタイムアウト値が極端に短い ★★☆☆

カテゴリ: Web設定 (php.conf) | 対象サーバ: Web (10.10.10.11) | ブランチ: lv1-q5

▶ ブラウザでの症状

503 Service Unavailable (タイムアウトによる)

▶ 原因

php.conf の ProxySet timeout が 30 (正) → 1 (誤) に変更。PHP-FPM の処理が 1 秒以内に完了しないと Apache がタイムアウトとして切断する。

```
- ProxySet connectiontimeout=5 timeout=1
+ ProxySet connectiontimeout=5 timeout=30
```

エラーハンドリング手順

1 エラーログを確認

```
# tail -20 /var/log/httpd/error_log
# → timeout やプロキシ関連のエラーが出ている
```

2 php.conf を確認

```
# cat /etc/httpd/conf.d/php.conf
# → timeout=1 という極端な値に気づく
```

3 timeout 値を修正

```
# vi /etc/httpd/conf.d/php.conf
# timeout=1 → timeout=30 に修正
```

4 Apache を再起動

```
# systemctl restart httpd
```

▶ 学びポイント

タイムアウト系のエラーは 503 として現れるが、ポート間違いの 503 とはログメッセージが異なる。「Connection refused」ならポート/起動の問題、「timeout」ならタイムアウト設定の問題。ログの内容を正確に読む力が切り分けの鍵。

問6：MariaDB の bind-address が localhost のみ ★★☆☆

カテゴリ: DB設定 | 対象サーバ: DB (10.10.10.13) | ブランチ: lv1-q6

▶ ブラウザでの症状

500 Internal Server Error (「Error establishing a database connection」)

▶ 原因

MariaDB の設定ファイルで `bind-address = 0.0.0.0` (正) → `bind-address = 127.0.0.1` (誤) に変更。MariaDB が localhost からの接続しか受け付けなくなるため、AP サーバ (10.10.10.12) からの接続が拒否される。

```
- bind-address = 127.0.0.1 ← localhost のみ
+ bind-address = 0.0.0.0 ← 全インターフェースで待ち受け
```

エラーハンドリング手順

1 AP サーバから DB に接続テスト

```
$ docker exec -it ap bash
# mysql -u wordpress-user -h 10.10.10.13 -p wordpress-db
# → "Can't connect to MySQL server" のエラー
```

2 DB サーバでリッスンアドレスを確認

```
$ docker exec -it db bash
# ss -tlnp | grep 3306
# → 127.0.0.1:3306 のみ → ここが問題
```

3 MariaDB の設定ファイルを確認・修正

```
# cat /etc/my.cnf.d/docker.cnf
# → bind-address = 127.0.0.1 を発見

# vi /etc/my.cnf.d/docker.cnf
# bind-address = 0.0.0.0 に修正
```

4 MariaDB を再起動

```
# systemctl restart mariadb
```

5 再度 AP サーバから接続テスト → 接続できれば OK

▶ 学びポイント

`bind-address` はサービスがどのネットワークインターフェースで接続を受け付けるかを制御する。
`127.0.0.1` = 自分自身のみ、`0.0.0.0` = 全ての IP から受付。 `ss -tlnp` でリッスンアドレスを確認する習慣は非常に重要。

問7：wp-config.php のパーミッションが 000 ★★☆

カテゴリ: パーミッション | 対象サーバ: AP (10.10.10.12) | ブランチ: lv1-q7

▶ ブラウザでの症状

500 Internal Server Error

▶ 原因

ap/Dockerfile で `chmod 000 /var/www/html/wp-config.php` が追加されている。PHP-FPM (apache ユーザー) が wp-config.php を読み取れないため、WordPress が設定を読み込めずにエラーになる。

```
+ RUN chmod 000 /var/www/html/wp-config.php ← 誰にも読めない権限
```

エラーハンドリング手順

1 AP サーバで WordPress のファイル権限を確認

```
$ docker exec -it ap bash
# ls -la /var/www/html/wp-config.php
# → ----- (000) と表示される → 誰にも読めない
```

2 適切な権限を設定

```
# chmod 644 /var/www/html/wp-config.php
# もしくは chmod 640 でも OK (所有者とグループが読める)
```

3 所有者も確認

```
# ls -la /var/www/html/wp-config.php
# → apache:apache であることを確認
```

4 ブラウザで再確認 → WordPress が表示されれば正解

▶ 学びポイント

ファイルのパーミッション（読み・書き・実行）は Linux の基本。 `ls -la` でパーミッションと所有者を確認する。Web アプリのファイルは、Web サーバのプロセスが動作するユーザー（ここでは apache）が読める必要がある。パーミッションは `chmod`、所有者は `chown` で変更する。

問8：wp-config.php の DB_HOST が存在しない IP ★☆☆

カテゴリ: AP設定 (wp-config.php) | 対象サーバ: AP (10.10.10.12) | ブランチ: lv1-q8

▶ ブラウザでの症状

500 Internal Server Error (「Error establishing a database connection」)

▶ 原因

wp-config.php の DB_HOST が 10.10.10.13 (正) → 10.10.10.99 (誤) に変更。存在しない IP に接続しようとしてタイムアウトまたは接続拒否される。

```
- define( 'DB_HOST', '10.10.10.99' ); ← 存在しないIP
+ define( 'DB_HOST', '10.10.10.13' ); ← DB サーバの正しいIP
```

エラーハンドリング手順

1 AP サーバで wp-config.php を確認

```
$ docker exec -it ap bash
# cat /var/www/html/wp-config.php | grep DB_HOST
# → DB_HOST が 10.10.10.99 になっている
```

2 その IP に接続テスト

```
# mysql -u wordpress-user -h 10.10.10.99 -p wordpress-db
# → 接続失敗 (ホストが存在しない)
```

3 正しい DB サーバの IP を確認

```
# docker-compose.yml を見れば DB は 10.10.10.13
# mysql -u wordpress-user -h 10.10.10.13 -p wordpress-db
# → 接続成功
```

4 wp-config.php を修正

```
# vi /var/www/html/wp-config.php
# DB_HOST を 10.10.10.13 に修正
```

▶ 学びポイント

DB 接続エラーの場合、まず wp-config.php の接続先情報（ホスト、ユーザー、パスワード、DB名）を一つずつ確認する。 `mysql` コマンドで手動接続テストを行い、どの情報が間違っているか切り分ける。

問9：PHP-FPM の `allowed_clients` が間違っている ★★☆☆

カテゴリ: AP設定 (www.conf) | 対象サーバ: AP (10.10.10.12) | ブランチ: lv1-q9

▶ ブラウザでの症状

503 Service Unavailable または 500 Internal Server Error

▶ 原因

PHP-FPM の `www.conf` で `listen.allowed_clients` が `10.10.10.11` (正・WebサーバのIP) → `10.10.10.99` (誤) に変更。Web サーバ (10.10.10.11) からの接続が PHP-FPM に拒否される。

```
- listen.allowed_clients = 10.10.10.99 ← 存在しないIPのみ許可
+ listen.allowed_clients = 10.10.10.11 ← Web サーバのIP
```

エラーハンドリング手順

1 Web サーバのエラーログを確認

```
$ docker exec -it web bash
# tail -20 /var/log/httpd/error_log
# → "Connection reset by peer" のようなエラー
```

2 AP サーバで PHP-FPM の設定を確認

```
$ docker exec -it ap bash
# grep "allowed_clients" /etc/php-fpm.d/www.conf
# → listen.allowed_clients = 10.10.10.99 を発見
```

3 PHP-FPM はリッスンしているか確認

```
# ss -tlnp | grep 8000
# → リッスンしている (ポートは正しい) → アクセス制御の問題
```

4 `allowed_clients` を修正

```
# vi /etc/php-fpm.d/www.conf
# listen.allowed_clients = 10.10.10.11 に修正
```

5 PHP-FPM を再起動

```
# systemctl restart php-fpm
```

▶ 学びポイント

`listen.allowed_clients` は PHP-FPM のファイアウォール的な機能。サービスが起動していてポートも開いているのに接続できない場合、アプリケーションレベルのアクセス制御を疑う。「起動している」「ポートが開いている」のに繋がらない → ミドルウェアの設定で拒否されている可能性。

問10：DB ユーザーに GRANT 権限がない ★★★

カテゴリ: DB設定 | 対象サーバ: DB (10.10.10.13) | ブランチ: lv1-q10

▶ ブラウザでの症状

500 Internal Server Error (「Error establishing a database connection」)

▶ 原因

db/entrypoint.sh で、`GRANT ALL PRIVILEGES` (正) の代わりに `CREATE USER` のみ実行 (誤)。ユーザーは作成されるが、`wordpress-db` データベースへのアクセス権限が一切付与されていない。

```
- GRANT ALL PRIVILEGES ON `wordpress-db`.* TO 'wordpress-user'@'10.10.10.12' IDENTIFIED BY 'wordpress-password';  
+ CREATE USER 'wordpress-user'@'10.10.10.12' IDENTIFIED BY 'wordpress-password'; ← ユーザー作成のみ (権限なし)
```

エラーハンドリング手順

1 AP サーバから DB 接続テスト

```
$ docker exec -it ap bash  
# mysql -u wordpress-user -h 10.10.10.13 -pwordpress-password wordpress-db  
# → "Access denied" → ユーザーは存在するが権限がない
```

2 DB サーバで権限を確認

```
$ docker exec -it db bash  
# mysql -u root  
MariaDB> SHOW GRANTS FOR 'wordpress-user'@'10.10.10.12';  
# → USAGE のみ (= 権限なし)
```

3 権限を付与する

```
MariaDB> GRANT ALL PRIVILEGES ON `wordpress-db`.* TO 'wordpress-user'@'10.10.10.12';  
MariaDB> FLUSH PRIVILEGES;
```

4 AP サーバから再度接続テスト

```
# mysql -u wordpress-user -h 10.10.10.13 -pwordpress-password wordpress-db  
# → 接続成功
```

5 ブラウザで再確認 → WordPress が表示されれば正解

▶ 学びポイント

MySQL/MariaDB のユーザー管理は「ユーザー作成」と「権限付与」が別の操作。 `CREATE USER` はユーザーを作るだけで、DB へのアクセス権は `GRANT` で別途付与する必要がある。 `SHOW GRANTS FOR` でユーザーの現在の権限を確認する。 `USAGE` は「権限なし（ログインのみ可能）」を意味する。